객체지향 프로그래밍 기초

메모리

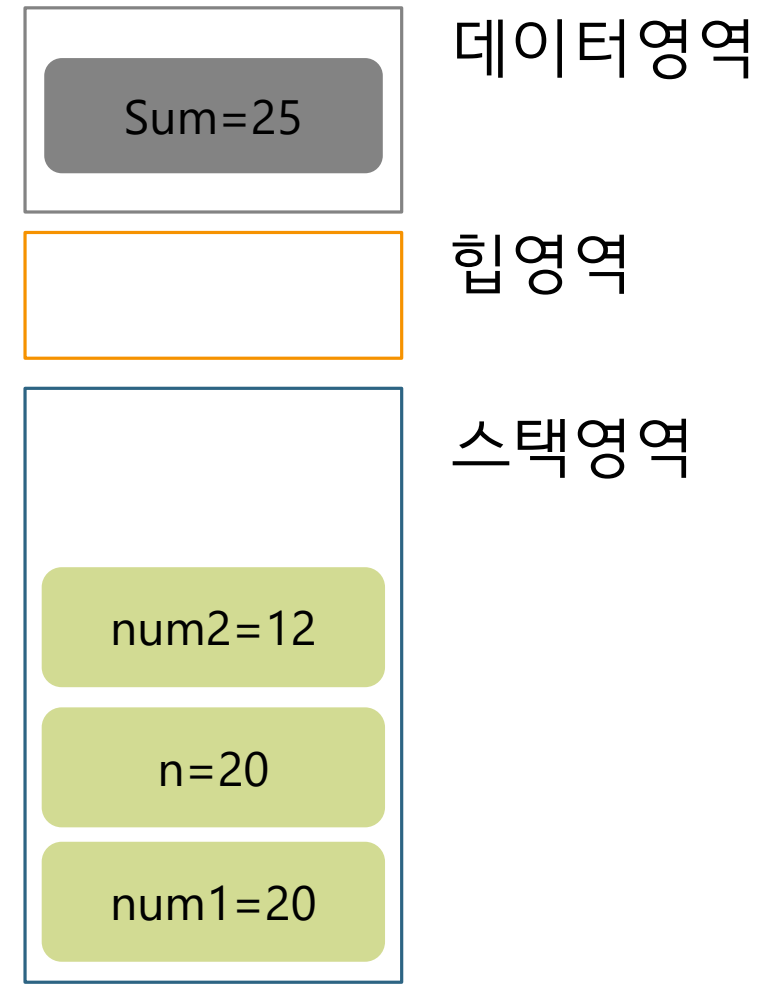
메모리의 구조

코드영역	실행할 프로그램의 코드가 저장되는 메모리 공간
데이터영역	전역변수와 static변수가 할당되는 메모리 공간 프로그램의 시작과 동시에 할당되어 종료시까지 존재한다.
힙영역	프로그래머가 원하는 시점에 할당/소멸하는 변수가 할당되는 메모리 공간
스택영역	지역변수와 매개변수가 할당 선언된 함수를 빠져나가면 소멸된다.

메모리의 구조

```
int sum = 25;
void func(int n)
{
        int num2 = 12;
}

int main()
{
        int num1 = 20;
        func(num1);
        num1++;
        func(num1);
        return 0;
}
```



메모리의 구조

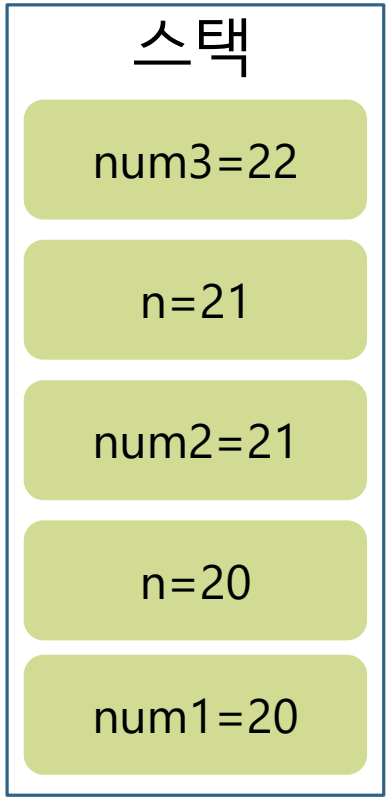
```
void func2(int n)
{
        int num3 = n+1;
}

void func1(int n)
{
        int num2 = n+1;
        func2(num2);
}

int main()
{
        int num1 = 20;
        func1(num1);
        return 0;
}
```

변수들이 메모리에 쌓이는 순서는?

✓ 함수의 호출 순서 :
main → func1 → func2



메모리의 동적 할당

- new 연산자로 지정하는 순간 할당되고 delete 연산자로 지정하는 순간 해제된다.
- 모든 연산은 포인터를 반환하므로 **동적 메모리 관리는 포인터를 사용한다.**
- **동적 메모리를 할당하고 모두 사용한 후에는 반드시 해제해야 한다.**

메모리의 동적할당

```
int *pt = new int;
//
delete pt;
```

- int형의 데이터 하나를 보관할 수 있는 크기의 동적 메모리 공간을 할당하고 할당된 메모리 블록의 주소를 반환
- 할당한 메모리를 모두 사용한 후 반드시 delete연산자를 사용하여 해제시킨다.

cf. malloc()
메모리 공간을 할당하는 기능만 가지고 있기 때문에 생성자를 호출하지 않는다.
할당할 대상의 정보를 무조건 바이트 크기 단위로 전달
반환형이 void형이므로 형변환이 필요

메모리의 동적할당

```
int main()
{
    int* pt = new int;

    *pt = 123;

    cout << *pt << endl;
    delete pt;

    return 0;
}
```

메모리의 동적할당

```
int *pt = new int;
```

VS

```
int num;
int * pt = &num;
```

메모리의 동적할당

잘못된 코드는 무엇인가? 이유는?

```
int main()
{
    int  *  ps = new int;
    delete ps;

    int jugs  = 5;
    int  *  pi = &jugs;
    delete jugs;

    return 0;
}
```

잘못된 코드

배열을 동적으로 할당

```
int * ptr = new int [10];
delete [] ptr;


char * sptr = new char [30];
delete [] sptr;
```

[예제1]

10개의 int형 배열을 동적메모리에 할당하고 1~10까지의 수를 저장한 후 평균과 합계를 출력하라.

배열을 동적으로 할당

```
int main()
{
    const int size = 10;
    int * arr = new int[size];

    for (int i=0; i<size; i++)
    {
            arr[i] = i + 1;
          cout << arr[i] << endl;
    }
    delete [] arr;

    return 0;
```

[예제2]

동적 메모리를 사용하여 이름과 주소를 저장하고 출력하시오.

new를 이용한 문자열 사용

- 프로그램이 1000개의 문자열을 읽어들여야 한다. 그 중 가장 긴 문자열은 79개의 문자로 되어 있고 나머지 문자열은 30개 미만의 문자로 되어 있다.
  - 문자열을 저장하기 위한 방법 1 : char형 배열을 사용

    -> 크기가 80인 char형 배열을 1000개 만들어야 한다. : 80000바이트(80KB)가 필요

    -> 대부분의 메모리 공간은 사용되지 않는다.
  - **방법 2 : char형을 지시하는 1000개의 포인터를 원소로 가지는 배열을 사용**

    -> 문자열마다 new를 사용하여 각 문자열의 크기에 맞는 메모리 블록을 대입하고 각 블록의 주소를 포인터 배열의 각 원소에 대입

    -> 메모리를 절약할 수 있다.

문자열의 동적할당

```
char * getname();

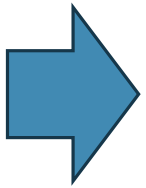
int main()
{
    char * name;
    name = getname();
    delete [] name;

   name = getname();
   delete [] name;
   return 0;
}
```

```
char * getname()
{
    char temp[80];
    cout << "이름을 입력하시오. : ";
    cin >> temp;
    char * pn = new char[strlen(temp) + 1];
    strcpy(pn, temp);
    return pn;
}
```

문자열의 동적할당

```
char animal[20] = “bear”;
const char * cat = “nabi”;
char * ps;
cout << animal << endl;
cout << cat << endl;
cout << ps << endl;
```



bear

nabi

쓰레기…

구조체의 동적 메모리 할당

```
struct Point{
        int x;
        int y;
};
int main()
{
    Point  *  ps;
    ps = new Point;
    cin >> ps->x >> ps->y;
    cout << ps->x << " " << ps->y << endl;
    delete ps;
    return 0;
}
```

구조체의 동적 메모리 할당

```
struct Point{
        int x;
        int y;
};

int main()
{
      Point  *pa;
      pa = new Point[10];
      for (int i=0; i<10;i++)
            cin >> pa[i].x >> pa[i].y;
      . . . .
      delete[] pa;
      return 0;
}
```

[예제]

구조체 배열을 동적 메모리를 이용하여 생성하고, 사용자로부터 영화정보를 입력받은 후 화면에 출력하는 프로그램을 작성하시오.

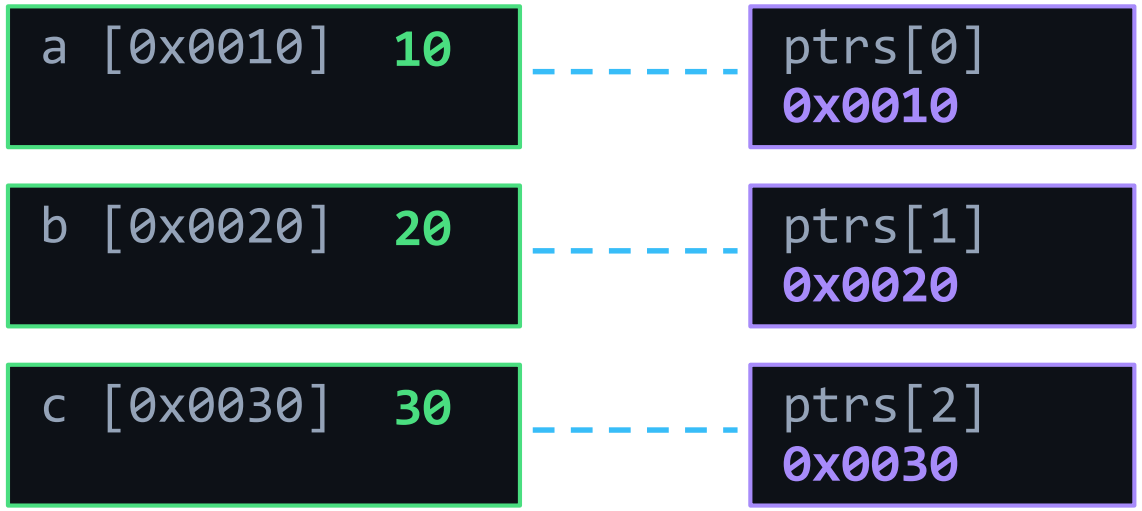


```
몇 편이나 저장하시겠습니까? 1
영화 제목:트랜스포머
영화 평점:8.3
========================
제목 평점
========================
트랜스포머 8.300000
========================
```

포인터 배열 (Array of Pointers)

int arr[3]
정수 배열 → 값 직접 저장

메모리 구조 시각화




```cpp
#include <iostream>
using namespace std;

int main() {
  int a=10, b=20, c=30;

  // 포인터 배열 선언
  int* ptrs[3];
  ptrs[0] = &a;  ptrs[1] = &b;
  ptrs[2] = &c;

  // 역참조로 값 출력
  for (int i=0; i<3; i++) {
    cout << *ptrs[i] << endl;
  }
}
```

동적 메모리 할당 — new / delete

스택 (Stack)

- 컴파일 타임에 크기 결정
- 함수 종료 시 자동 해제
- 크기 변경 불가
- 빠르지만 유연성 낮음

힙 (Heap)

- 런타임에 크기 결정 가능
- 프로그래머가 직접 해제
- 크기 자유롭게 변경
- 유연하지만 관리 필요

```
int* arr = new int[5];

for (int i=0; i<5; i++)
      arr[i] = i *10;

delete[] arr;
arr = nullptr;          // 배열은 delete[]
```

실전: 구조체 + 포인터 배열 + 동적 메모리


```
struct Player {
  int id;
  int score;
};

int main() {
  int n = 3;  // 플레이어 수
  Player* players = new Player[n];

  Player* ptrs[3];

  for (int i=0; i<n; i++) {
    players[i].id = i+1;
    players[i].score = (i+1)*100;
    ptrs[i] = &players[i];
  }

  for (int i=0; i<n; i++)
    cout << ptrs[i]->score;

  delete[] players;
  return 0;
}
```

01 **struct Player 선언**
id, score 필드를 가진 플레이어 구조체 정의

02 **new Player[n]**
힙에 n개의 Player 구조체 배열 동적 할당

03 **포인터 배열 ptrs[]**
각 Player 구조체의 주소를 포인터 배열에 저장

04 **ptrs[i]->score**
화살표(->)로 포인터를 통한 멤버 접근

05 **delete[] players**
힙에 할당된 배열 해제 — 필수!

핵심 정리 & 주의사항

개념	선언 방법	사용 예	주의점
포인터	`int* p = &x;`	`*p = 42;`	nullptr 초기화 권장
포인터 배열	`int* arr[N];`	`arr[i] = &x;`	배열 크기 N 고정
동적 단일 할당	`int* p = new int;`	`*p = 10;`	delete p; 필수
동적 배열 할당	`int* a = new int[N];`	`a[i] = i;`	delete[] a; 필수
구조체 포인터	`Player* p = new Player;`	`p->score = 100;`	delete p; 필수

✕ 흔한 실수

- delete 없이 포인터 재할당 → 메모리 누수
- 배열에 delete ([] 없이) 사용 → 미정의 동작
- 해제된 포인터 재사용 → 댕글링 포인터

✓ Best Practice

- 포인터 선언 즉시 nullptr 초기화
- delete 후 반드시 p = nullptr 대입
- 배열 해제는 항상 delete[] 사용

예제

```
int main() {
	int n = 5; // int* 포인터 배열을 동적할당
	int **arr = new int*[5];
	for (int i = 0; i < n; i++) {
		arr[i] = new int;
		*arr[i] = (i + 1) * 10;
	}
	for (int i = 0; i < n; i++)
		cout << "arr[" << i << "] ->" << arr[i] << " -> " <<* arr[i] << endl;

	for (int i = 0; i < n; i++)
		delete arr[i];
	delete [] arr;
	return 0;
}
```

포인터 배열과 동적 메모리

```
struct Point {
    int x;  int y;
};


Point* pt[100] = { NULL };

pt[i] = new Point;

pt[i]->x = 10;
pt[i]->y = 20;
```

```
struct Point {
    int x, int y;
};

Point point[3] = { {0,1}, {2,3}, {4,5} };
Point *pt = &point[0];
Point *ptarray[3];
ptarrya[0] = pt;

Point* pt[3] = { NULL };

ptarray[0] = new Point;

ptarray[0]->x = 10;
ptarray[0]->y = 20;
```

예제

```
struct Character {
        int id;
        int hp;
 };
void changeTarget(Character** p, Character* arr) {
// p가 가리키는 포인터 변수의 원본 주소값을 배열의 3번째 원소(index 2) 주소로 변경
        *p = &arr[2];
}
int main() {
        Character team[3] = { {1, 100}, {2, 300}, {3, 500} };
        Character* target = &team[0]; // 함수 호출 시 target 포인터 변수의 주소(&)를 넘
겨 원본을 수정하고자 함
        changeTarget(&target, team);
        cout << target->hp << endl;
        return 0;
}
```

[예제]

구조체 배열 포인터를 동적 메모리를 이용하여 생성하고, 사용자로부터 영화 정보를 입력받은 후 화면에 출력하는 프로그램을 작성하시오.



```
몇 편이나 저장하시겠습니까? 1
영화 제목:트랜스포머
영화 평점:8.3
========================
제목 평점
========================
트랜스포머 8.300000
========================
```

변수의 종류

- 변수는 선언되는 위치에 따라서 '전역변수 ' 와 '지역변수 ' 로 나뉜다.
  - 메모리상에 존재하는 기간
  - 변수에 접근할 수 있는 범위

지역변수

- **'지역'이란 중괄호에 의해 형성되는 영역을 뜻한다.**
- **중괄호 내에 선언되는 변수는 모두 지역변수이다.**

```
int SimpleFunc()
{
    int num = 3;
    cout << num << endl;

    num = 200;
    cout << num << endl;

    return 0;
}
```

- 초기화하지 않으면 불필요한 값을 갖는다.
- 함수를 호출할 때마다 변수는 초기화된다.
- 지역변수는 해당 지역을 벗어나면 자동으로 소멸된다.
- 지역변수는 선언된 지역 내에서만 유효하므로 선언된 지역이 다르면 이름이 같아도 된다.

지역변수

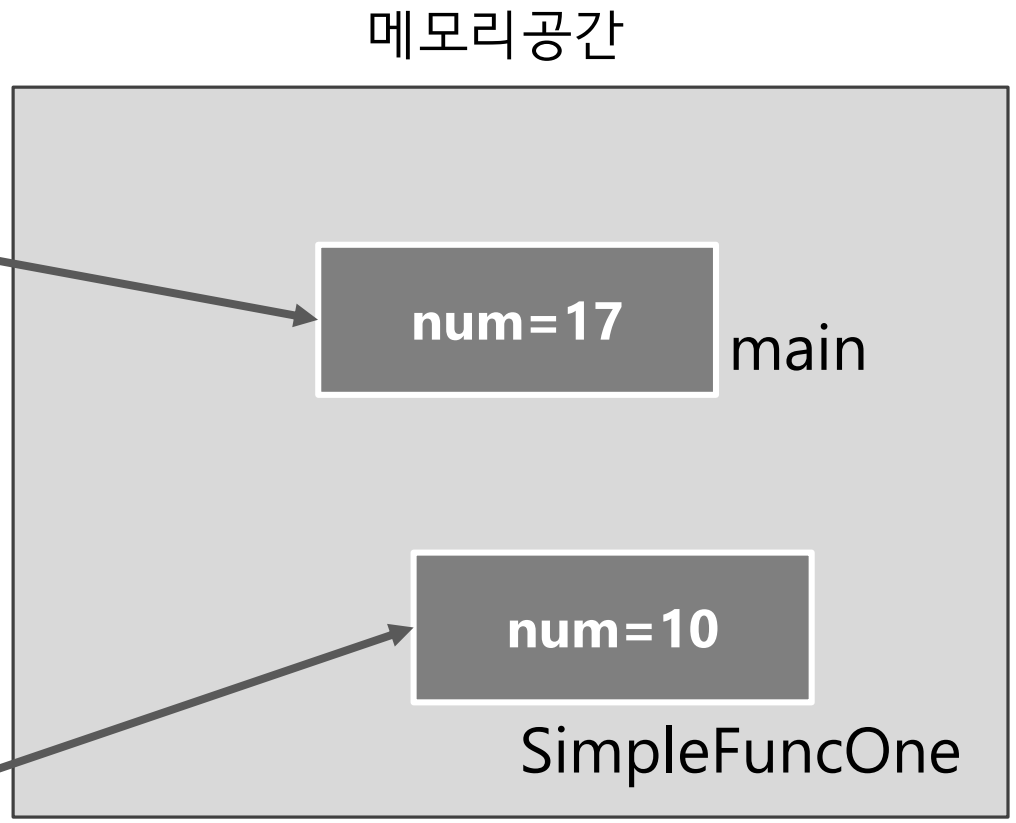
```
int SimpleFuncOne()
{
   int num = 10;
   num++;
   cout << "SimpleFuncOne:" << num << endl;
   return 0;
}
int SimpleFuncTwo(void)
{
   int num = 20, ref = 30;
   num++; ref++;
   cout << "Two:" << num << " , " << ref << endl;
   return 0;
}
int main()
{
   int num = 17;
   SimpleFuncOne();
   SimpleFuncTwo();
   cout << "main:" << num << endl;
   return 0;
}
```

지역변수

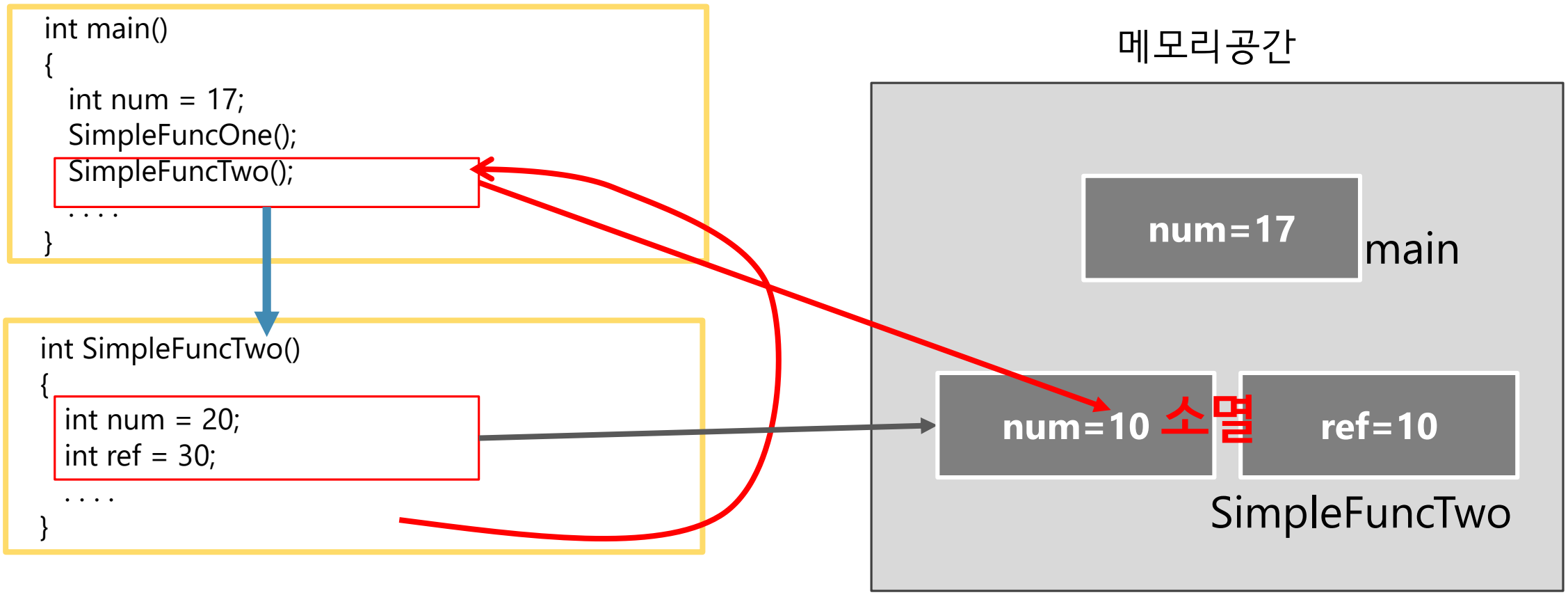
```
int main()
{
  int num = 17;
  . . . .
}
```

```
int main()
{
  int num = 17;
  SimpleFuncOne();
  . . . .
}
```

```
int SimpleFuncOne()
{
  int num = 10;
  . . . .
}
```



지역변수



지역변수는 해당 선언문이 실행될 때 메모리 공간에 할당되었다가,
선언문이 존재하는 함수가 반환을 하면 메모리 공간에서 소멸된다.

지역변수

```
int main()
{
    int x;

    for (x = 1; x < 5; x++)
    {
        int count = 0;
        count ++;
        cout << "x = " << x << endl;
        cout << "count = " << count << endl;
    }
    cout << "for문 밖의 x = " << x << endl;
    return 0;
}
```

지역변수

```
int main()
{
    int x;

    for (x = 1; x < 5; x++)
    {
        int x = 0;
        x ++;
        cout << "x = " << x << endl;
    }
    cout << "for문 밖의 x = " << x << endl;
    return 0;
}
```

매개변수

함수를 정의할 때 선언하는 매개변수도 지역변수의 일종이다.

- 선언된 함수 내에서만 접근 가능하다.
- 선언된 함수가 반환되면, 지역변수와 마찬가지로 소멸이 된다.

전역변수

- **전역변수는 함수 밖에서 선언한 변수이다.**

```
#include <iostream>
using namespace std;

int x = 100;

int main()
{
    cout << "전역변수 x = " << x  << endl;

    x = 200;
    cout << "x = " << x << endl;

    return 0;
}
```

- 전역변수의 유효범위는 선언된 위치부터 프로그램 끝까지이다.
- 전역변수는 초기화하지 않아도 자동으로 0으로 초기화된다.
- 프로그램의 시작과 동시에 메모리 공간에 할당되어 종료 시까지 존재한다.
- 프로그램 전체 영역 어디에서든 접근이 가능하다.

전역변수

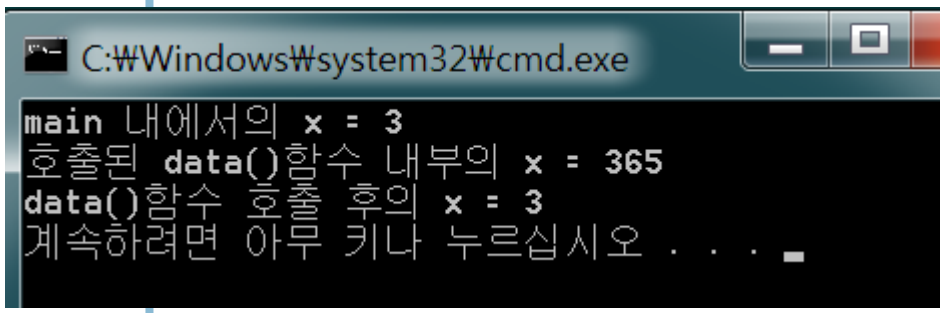
```
void data();

int x = 7;

int main()
{
    int x = 3;
    cout << "main내에서의 x = " << x << endl;
    data();
    cout << "data() 함수 호출 후의 x = " << x << endl;
    return 0;
}

void data()
{
    int x = 365;
    cout << "호출된 data()함수 내부의 x = " << x << endl;
    x = 100;
}
```

- 전역변수와 동일한 이름의 지역변수가 선언되면 해당 블록 내에서는 전역변수가 가려지고, 지역변수로의 접근이 이루어진다.



전역변수

```
int Add(int val);
int num = 1;

int main()
{
    int num = 5;
    cout << “Add(3)함수 결과값 = “ << Add(3) << endl;
    cout << “num = “ << num << endl;
    return 0;
}

int Add(int val)
{
    int num = 9;
    num += val;
    return num;
}
```

```
add(3)함수 결과값 = 12
num = 5
```

정적변수(static)

- 지역변수에 static 을 붙이면 선언된 함수 내에서만 접근이 가능하다.
- 한 번만 초기화되고 프로그램 종료시까지 메모리 공간에 존재한다.

```
void Simple()
{
    static int num1 = 0;
    int num2 = 0;
    num1++; num2++;
    cout << "static : " << num1 << " local : " << num2 << endl;
}

int main()
{
    int i;
    for (i = 0; i < 3; i++)
        Simple();
    return 0;
}
```

```
static: 1 local: 1
static: 2 local: 1
static: 3 local: 1
계속하려면 아무 키나 누르십시오 . . .
```

정적변수(static)

```
void data();

static int x = 7;

int main()
{
    cout << "main 함수 내의 x = " << x << endl;
    data();
    cout << "data()함수 호출 후의 x = " << x << endl;
    return 0;
}

void data()
{
    x = 100;
    cout << "호출된 data() 함수의 x = " << x << endl;
    x = 200;
}
```

```
Main 함수 내의 x : 7
호출된 data() 함수의 x = 100
data()함수 호출 후의 x = 200
```

[예제1] 전역변수의 사용

아래는 세 개의 정수값을 입력받은 후, 입력받은 값의 누적 합을 계산하는 프로그램이다. 전역변수를 AddToTotal()함수의 정적변수로 변경하라.

```
int total = 0;
int AddToTotal(int num)
{
    total += num;
    return total;
}
int main()
{
    int num, i;
    for (i=0; i<3; i++)
    {
        cout << "입력 : " << i + 1 << endl;
        cin >>num;
        cout << "누적 합계 : " << AddToTotal(num) << endl;
    }
    return 0;
}
```

extern 변수

SimpleFn.cpp

```
int Count = 0;
int Add(int a, int b)
{
    int sum;
    sum = a+b;
    return sum;
}
```

SimpleFnMain.cpp

```
#include <stdio.h>
extern int Count;
extern int Add(int, int);

int main()
{
    int x=3, y=4, z;
    Count++;
    z = Add(x, y);
    cout << z;
    return sum;
}
```

SimpleFn.h

```
extern int Count;
extern int Add(int, int);
```